

UDP/TCP Echo Application

Computer Networks project, A.A. 2025/26

Feri Alessandro, 7081870

1. Introduction

The goal of this project was to develop and test a small network application using UDP and TCP. The application can run as a server or as a client. The server opens a port chosen at run time, receives data, and replies with an echo message. The reply can also be modified before it is sent back. The client sends a chosen number of messages and waits for the replies.

2. Program Design

The program has two main modes:

```
server <protocol> <port>
client <protocol> <host> <port> <message>
```

The supported protocols are `udp` and `tcp`. The client option `--count` selects how many messages are sent. The server option `--transform` selects how the echo is changed. The possible values are `none`, `upper`, and `reverse`.

For UDP, one datagram is used as one application message. This is natural because UDP preserves datagram boundaries. The server receives a datagram, applies the transform if needed, and sends one datagram back to the sender.

For TCP, the situation is different. TCP is a stream protocol, so it does not preserve message boundaries. A single read can receive only part of a message, or more than one message together. For this reason the application defines its own framing format:

```
[4-byte big-endian payload length][payload bytes]
```

The receiver first reads the 4-byte length field, then it reads exactly that number of payload bytes. In this way TCP messages are reconstructed at the application level.

3. RTT and Statistics

The program also implements an optional RTT measurement with `--rtt`. When this option is used, the client adds an 8-byte timestamp at the start of the payload. The server echoes this timestamp without changing it and applies transformations only to the user message after the timestamp.

The timestamp format is NTP64. It contains 32 bits for seconds since 1900-01-01 UTC and 32 bits for the fractional part of the second. I used NTP64 because the operating system gives UTC/Unix time directly, while I did not have a TAI clock available. This keeps the implementation portable.

The client prints the RTT for each valid reply and then prints a short summary:

```
Avg RTT
Max RTT
Sent
Received
Unanswered
```

Unanswered means that the client sent an application message but did not receive a valid reply. For UDP this can mean packet loss or timeout. For TCP this is an application-level missing reply, because TCP hides transport losses with retransmissions.

4. Test Setup

The application was tested with a server running on an Amazon EC2 instance and a client running from my machine. The EC2 server was in Frankfurt, Europe. I opened UDP port 8888 and TCP port 9999 in the EC2 security group/firewall, installed the server program on the instance, and ran it there for the tests. The server IP was 52.29.154.138. Each main test sent 1000 messages with RTT enabled. For each test case I started two client runs in parallel, so the server had to answer two clients at the same time.

Example commands:

```
cargo run -- server udp 8888 --rtt
cargo run -- client udp 52.29.154.138 8888 ciao --count 1000 --rtt

cargo run -- server tcp 9999 --rtt
cargo run -- client tcp 52.29.154.138 9999 ciao --count 1000 --rtt
```

I also tested TCP with `--no-delay`, which enables `TCP_NODELAY` and disables Nagle's algorithm. This was useful to see if small echo messages had different RTT behavior.

5. Results

The basic UDP and TCP runs are shown in the next table.

Protocol	Run	Sent	Received	Unanswered	Avg RTT	Max RTT
UDP	1	1000	1000	0	20.012 ms	25.005 ms
UDP	2	1000	1000	0	19.327 ms	23.766 ms
TCP	1	1000	1000	0	20.227 ms	77.253 ms
TCP	2	1000	1000	0	19.234 ms	75.241 ms

Both protocols completed the tests with all replies received. The average RTT was similar for UDP and TCP, around 19-20 ms. In these runs TCP had higher maximum RTT values, but these were isolated outliers.

The TCP tests with and without `TCP_NODELAY` are shown below.

TCP mode	Run	Sent	Received	Unanswered	Avg RTT	Max RTT
Default	1	1000	1000	0	20.227 ms	77.253 ms
Default	2	1000	1000	0	19.234 ms	75.241 ms
No delay	1	1000	1000	0	20.091 ms	24.458 ms
No delay	2	1000	1000	0	19.233 ms	26.423 ms

Disabling Nagle's algorithm did not change the average RTT in a strong way. However, in these measurements it reduced the worst observed TCP RTT. This result should not be considered general, because it depends on the network path and on the operating system behavior.

6. Echo Transform Tests

The server was also tested with payload transformations. The message sent by the client was `ciao`.

Protocol	Transform	Received payload	Sent	Received	Unanswered
UDP	upper	CIAO	1000	1000	0
UDP	reverse	oaic	1000	1000	0
TCP	upper	CIAO	1000	1000	0
TCP	reverse	oaic	1000	1000	0

These tests show that the echo modification works for both protocols. They also show that the RTT timestamp is handled correctly, because the transformation is applied only to the message part and not to the timestamp bytes.

7. Interoperability

Interoperability with other groups was planned, but it was not possible to complete it because no external group was available for testing. To make the application easier to test with other implementations, the command-line usage and the TCP framing format are described in this report.

UDP interoperability is simple because one datagram is one message. TCP interoperability requires the peer to use the documented 4-byte big-endian length prefix. Without this framing rule, two TCP programs may connect but they will not agree on where each application message starts and ends.

8. Conclusion

The project satisfies the main requirements. The application can send and receive UDP datagrams, open TCP connections, work as a client or server, use run-time ports, send a configurable number of messages, and echo replies with optional modifications.

The optional RTT feature was also implemented and tested. The AWS tests showed no unanswered messages in the recorded 1000-message runs. UDP and TCP had similar average RTT values, while TCP showed some larger outliers in the default configuration. With `TCP_NODELAY`, the largest TCP RTT values were lower in these tests.

Overall, the application is small but it shows the main difference between UDP and TCP. UDP gives message boundaries directly, while TCP needs an application-level framing protocol.